Real-Time Transit Reliability Lakehouse

Deriving arrival times that no agency publishes, and grading the predictions that riders actually see

Aatish Lobo Borna [last name]

MSDS 682 — Data Stream Processing — Summer 2026

Contents

1. Problem, user, and result

1.1 The problem

A rider wants to know whether a route is reliable. An analyst wants to compare operators. Both questions reduce to one number — how late is the vehicle, actually — and that number is surprisingly hard to obtain.

Transit agencies publish two live feeds through the GTFS-Realtime standard:

- **TripUpdates** — *predictions*: "the vehicle on trip X will reach stop 22 at 15:07."
- **VehiclePositions** — *GPS observations*: "vehicle 4821 is at this coordinate, now."

Neither feed ever states *"vehicle 4821 arrived at stop 22 at 15:09."* The fact of arrival is not published by anyone. It must be **derived**.

This matters more than it first appears. Delay is defined as `actual - scheduled`, and the scheduled time is published and exact. Therefore **every error in the derivation of "actual" transfers directly into the delay figure**, and no downstream processing can detect or correct it. Worse, the danger is *bias* rather than noise: random error averages out across a route, while a systematic offset shifts an entire distribution invisibly.

1.2 Target user and intended result

Primary user: a transit rider deciding whether to trust the arrival estimate in their app. **Secondary user:** an analyst comparing operators, or an agency auditing its own prediction quality.

Delivered result: a quantified answer to a question no operator publishes —

> When an agency says the bus arrives in N minutes, how wrong is it?

reported by lead time, route, and hour of day, together with a bounded machine learning model that measurably improves those predictions.

1.3 Why the answer is credible

The result is meaningful only because its two halves come from **independent sources**:

- the **prediction** comes from TripUpdates — the agency's forecast;
- the **actual arrival** is derived by us from VehiclePositions — observing the vehicle report `STOPPED_AT`.

The two feeds share no data and no logic. A tempting alternative — deriving arrivals from the last prediction issued before a vehicle passes, a technique known as *prediction settlement* — would have made this comparison measure the agency against itself, producing an impressively small error that means nothing. That trap, and its avoidance, is discussed in Section 7.2.

2. Data source and classification

2.1 Source

Name	511 SF Bay Open Data — GTFS-Realtime transit feeds
Owner	Metropolitan Transportation Commission (MTC)
Portal	https://511.org/open-data/transit
Format	GTFS-Realtime (Protocol Buffers, proto2)
Coverage	All Bay Area operators, one consolidated regional feed (`agency=RG`)
Access	Free API key; 60 requests per 3,600 seconds

Data provided by 511.org (Metropolitan Transportation Commission), http://www.511.org.

Full documentation of schema, rights, and limitations is in `DATA_SOURCE.md`.

2.2 Classification

Hybrid, with a real-time core. The GTFS-Realtime protobuf feeds are polled continuously and streamed through Kafka. The project's *submitted review path* is a deterministic replay of archived feed data, which requires no API key and produces identical results on every run.

2.3 Rate limits and their consequence

The default quota of 60 requests per hour, across two feed types, caps polling at one cycle per 120 seconds — exactly at the limit, with zero headroom. The budget is enforced **client-side** by a sliding-window limiter rather than by reacting to HTTP 429 responses, because a throttled token halts archiving and GTFS-Realtime has no history endpoint: minutes lost are lost permanently.

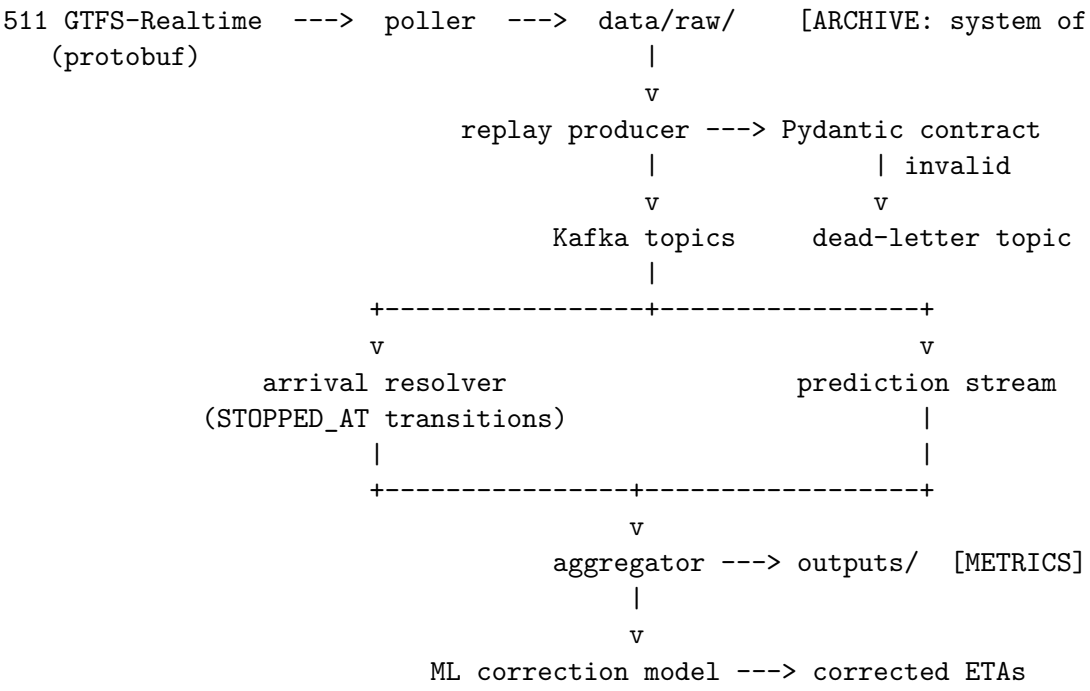
This cadence has a direct and unavoidable effect on data quality. The poll interval is the **noise floor of every derived arrival time** — an arrival cannot be observed more precisely than the interval that sampled it. It is therefore recorded on every archived row as `poll_interval_s`, so that a future change in cadence does not silently invalidate older data.

2.4 Licensing and why the repository is private

The data is governed by the 511 Data Disseminator Agreement. Section 1 grants a broad licence to use, copy, distribute, store, and create derivative works. However, **Section 2(c)** requires securing written acceptance of the agreement's terms from any third party *before* providing them the data.

The committed replay sample contains verbatim 511 protobuf payloads — the Provided Data itself, not a derivative work. Publishing it in a public repository would distribute that data to anyone, with no acceptance secured. The repository is therefore **private**, with access granted individually. Section 5(b) attribution requirements are met in the README, in `DATA_SOURCE.md`, and programmatically in the generated output artifacts.

3. Architecture



3.1 Two structural decisions

The archive is the system of record; Kafka is transport. Topics are configured with 24-hour retention deliberately. GTFS-Realtime cannot be re-fetched, so anything not written to disk is lost permanently. Treating a Kafka topic as durable storage is a common and expensive error — retention silently expires the data, and there is no way to recover it.

Live and replay share one code path. The obvious shortcut is to have the live poller publish directly to Kafka and treat replay as a separate testing utility. That produces two code paths which drift apart, and the one a reviewer runs is the one that receives less attention. Instead, the poller *only* writes to disk and everything downstream *only* reads from disk. Live operation and replay differ solely in whether new files continue to appear. A reviewer therefore exercises the genuine pipeline rather than a simulation of it.

3.2 Ingest

The poller fetches both feeds every 120 seconds. Its ordering is deliberate: **raw bytes are written to disk before any decode is attempted**. The two failure modes are not equally recoverable — a decoding bug discovered later can be repaired by reprocessing the archive, whereas a poll never taken is gone forever. A decode failure quarantines the payload rather than discarding it.

Additional behaviours: stale-feed detection via payload hash and header timestamp; single-instance enforcement via a lock file; connection-failure retry that charges every attempt to the rate budget; and redaction of the API key from all log output.

3.3 Streaming

Four Kafka topics, created explicitly rather than auto-created:

Topic	Partitions	Retention	Purpose
`gtfsrt.trip_updates.v1`	3	24 h	agency predictions
`gtfsrt.vehicle_positions.v1`	3	24 h	GPS observations
`gtfsrt.dead_letter.v1`	1	7 d	records failing validation
`transit.arrival_events.v1`	3	7 d	derived arrivals

Partition count was chosen, not defaulted. Partition assignment is computed from the message key, so increasing the count later changes which partition a key maps to — splitting a key's history across partitions and breaking ordering *retroactively*, including for data already written.

Cleanup policy is `delete`, never `compact`. Compaction retains only the most recent message per key. These topics are an event log: the sequence of observations over time *is* the data, and the arrival resolver works precisely by reading state transitions across consecutive observations. Compaction would destroy exactly what it consumes.

4. The event contract

Contracts are defined with Pydantic v2 in `streaming/contracts.py`. Every record is validated before publication; failures are routed to a dead-letter topic rather than crashing the consumer or being silently dropped.

4.1 Why a contract exists

The producer and consumers are written by the same team, so validation could in principle be skipped. It is not, for three reasons:

1. **It marks a boundary of ownership.** Once a record is on a topic, any number of consumers may read it. The contract is the only statement of what those consumers may assume.
2. **It detects drift.** If the decoder gains a field the contract does not declare, `extra="forbid"` raises immediately rather than allowing the mismatch to reach a consumer that ignores it.
3. **It makes bad records routable rather than fatal.**

4.2 The invariant the contract must not break

Pydantic **v2** is a hard requirement, not a preference. Version 1 coerced `None` to a field's declared default. Given that 44.2 percent of records carry no delay value (Section 6.1), a default of zero would have relabelled roughly 54,000 records per poll cycle as "exactly on time" — undoing the project's founding invariant at the one boundary specifically built to protect it. Strict mode is enabled for the same reason: a value arriving as the wrong *type* signals an upstream change and must fail rather than be silently repaired.

4.3 Partition key

```
key = f"{service_date}:{trip_id}"
```

Kafka guarantees ordering only *within* a partition and routes by key hash. Keying on the trip therefore guarantees that every observation of one trip lands in one partition, in order — a precondition for the arrival resolver, which detects arrivals by observing state transitions across consecutive polls.

`service_date` is included because `trip_id` is unique only *within* a service date; the same identifier recurs daily. Keying on `trip_id` alone would force different days of the same scheduled trip into one partition, coupling them needlessly and skewing load.

5. Deriving arrivals

5.1 Three methods, one implemented

Method	Signal	Weakness
A — position state	`current_status == STOPPED_AT` at a stop	not published by all operators
B — geofence	nearest approach of GPS trail to stop	measures closest approach, biased early
C — prediction settlement	last prediction before passing	inherits agency error; enables leakage

Method A is implemented. Method C is effectively unavailable on this feed (Section 6.1) and would compromise the evaluation. Method B requires GTFS-Static stop coordinates, a separate data source outside the scope taken.

5.2 The arrival is the first sighting, not the last

A vehicle waiting at a stop reports `STOPPED_AT` on *every* poll for the duration:

```
poll 1:  approaching stop 25
poll 2:  STOPPED_AT stop 25     <- the arrival
poll 3:  STOPPED_AT stop 25     <- still waiting
poll 4:  approaching stop 26
```

Taking the last such observation would measure **departure**; taking an intermediate one measures nothing meaningful. Only the first is the arrival. The resolver therefore maintains per-trip state, and this requirement is what makes the partition-key decision of Section 4.3 load-bearing rather than decorative.

5.3 Provenance on every row

Each derived arrival records not only *what* but *how*:

```
{
  "service_date": "20260806", "trip_id": "SF:12053041_M11",
  "stop_sequence": 47, "stop_id": "13550", "route_id": "SF:1",
  "actual_arrival_ts": "2026-08-06T21:52:43+00:00",
  "arrival_method": "stopped_at", "arrival_confidence": "high",
  "poll_interval_s": 120, "resolver_version": "1.0.0"
}
```

Because resolver availability varies by operator, recording only the final timestamp would allow *which agency a vehicle belongs to* to silently determine *how accurate its arrival time is*. Any later analysis would then discover "agency effects" that are in fact artifacts of our own code. Recording the method makes that variation visible and filterable. `poll_interval_s` travels with each row as its intrinsic error bar.

Records that cannot produce an arrival are **counted, not discarded** — the counts are themselves the measurement of resolver coverage per operator.

6. Findings from the live feed

All figures below are measured, not assumed. Source: `profiling/profile_feed.py` and the evaluation harness.

6.1 Feed characteristics

The regional feed carries 28 operators, not the four commonly named (BART, Muni, AC Transit, Caltrain).

44.2 percent of StopTimeUpdate rows carry no delay value at all — 54,638 of 123,735 in one sample. Nineteen of 28 operators never populate the field; AC Transit produced 23,469 consecutive rows with delay absent. Because GTFS-Realtime is proto2, a naive read returns `0` for both "absent" and "exactly on time". Implemented the obvious way, this project would have reported those 54,638 records as perfectly punctual — a fabricated on-time spike, entirely plausible in appearance, corrupting every downstream metric.

Only 15 of 28 operators publish `current_status`, the field method A requires. Muni populates it 99.2 percent of the time; Caltrain, 0 percent.

Settled predictions are effectively unavailable — only 4 of 28 operators emit `uncertainty = 0`, at negligible volume. Muni never publishes uncertainty at all, a fact independently reconfirmed during model training when the column was automatically dropped as entirely absent.

Seven operators run trips that revisit the same `stop_id` (Emery Go-Round: 23 of 29 trips), which is why the grain is `(service_date, trip_id, stop_sequence)` rather than `stop_id`.

A single payload spans multiple service dates. One observed poll contained trips dated 20260805 (58,971 rows), 20260806 (263 rows, already past midnight), and 2,858 rows with no date at all; vehicle positions in the same payload reached back eight days. Service date is a property of a *row*, not of a payload — which is why the raw archive partitions on ingest date instead.

6.2 Sampling limits

Vehicles dwell at stops for seconds while we sample every 120 seconds. Measured on SF Muni, **90.3 percent of captured stop events appear in exactly one poll**, and overall capture is approximately **21 percent** of stop events.

This produces three distinct effects, which must not be conflated:

1. **Coverage.** Most stop events are missed entirely. A sample-size limitation, not an accuracy one.
2. **One-sided offset.** A vehicle is observable as `STOPPED_AT` only between arriving and departing, so a derived timestamp falls at or *after* the true arrival — never before. Derived arrivals are biased **late**.
3. **Selection bias.** Capture probability scales with dwell time, so long-dwell stops — terminals, layovers, timepoints — are over-represented. Median observed dwell among multi-poll sightings is 353 seconds.

Consequence: effects 2 and 3 both push measured "actual" arrivals later, making agencies appear more optimistic than they may be. Every bias figure in this report should be read as an **upper bound**, not a point estimate.

7. Results

7.1 Prediction accuracy (SF Muni, 25,501 prediction-arrival pairs)

lead time	n	bias (s)	median abs err (s)	p90 (s)	within 60 s	within 180 s
0–2 min	509	-11	13	52	92.7%	100.0%
2–5 min	4,603	-46	50	112	59.5%	98.4%
5–10 min	4,897	-66	70	172	43.3%	91.2%
10–20 min	8,919	-98	102	257	31.1%	77.0%
20+ min	6,573	-142	146	375	23.0%	59.3%

Short-horizon predictions are excellent. Under two minutes out, the median error is 13 seconds and 92.7 percent land within a minute.

Accuracy degrades steadily with horizon. By 20 minutes out, only 23 percent fall within a minute and the median error approaches 2.5 minutes.

The bias is negative at every horizon and grows. Negative means the agency predicts *earlier* than observed: vehicles arrive later than promised, by an average of 142 seconds at long horizons — subject to the upper-bound caveat of Section 6.2.

Two statistics are reported deliberately. Median absolute error describes typical magnitude; **mean signed error is the bias**, and bias is what matters. Random error averages out across a route; systematic optimism does not, and it is what causes riders to miss vehicles.

One methodological exclusion: a "prediction" issued *after* the vehicle had already arrived is a correction, not a forecast. Scoring such pairs would count hindsight as foresight. Forty pairs were excluded, and the exclusion is counted rather than silent.

7.2 The bounded AI element

Task. Predict the residual `actual_arrival - agency_predicted_arrival`, then apply it as a correction to the agency's ETA. Predicting zero is identical to trusting the agency, making the baseline comparison exact.

Model. `HistGradientBoostingRegressor`, seven features, 1,008,746 training rows, 336,529 test rows, split **strictly forward in time**.

	MAE (s)	RMSE (s)	bias (s)	within 60 s
baseline: trust the agency	195.2	381.9	-154.0	31.0%
baseline: add global mean	215.1	358.2	+78.5	14.8%
baseline: add mean per horizon	206.7	359.2	+74.8	22.4%
model	**166.6**	**305.6**	**+43.7**	**32.1%**

14.7 percent lower error than the agency's own prediction, beating all three baselines. Improvement by horizon ranges from 12.5 percent (20+ min) to 19.9 percent (5–10 min).

A result worth noting: **both naive bias corrections perform worse than trusting the agency.** Residuals are heavily right-skewed (mean +213 s, median +111 s), so adding the mean

overcorrects the typical case. The naive application of Section 7.1's finding would have made predictions worse; the model earns its place by learning a horizon-, hour-, and route-dependent correction.

Leakage defences

This is deliberately a **prediction-correction model**, and that is declared rather than obscured. Using an agency forecast as a feature is normally leakage. It is legitimate here for a structural reason: **labels come from VehiclePositions, features from TripUpdates** — different feeds, so the label cannot contain the feature.

Three specific defences:

`lead_time` is excluded. Defined as `actual_arrival - issued`, it is used legitimately in evaluation and would be a natural addition to the feature list. It contains the target, and would produce a spectacular score and a model incapable of running in production. Asserted by test.

The split is temporal, not random. Random splitting leaks twice: the same trip contributes many rows which would scatter across both sides, and later observations would train a model tested on earlier ones. Both inflate the score invisibly.

A feature that improved the score was removed. Day-of-week reduced MAE by 10 seconds (156.8 s versus 166.6 s) and was dropped, because `dow = 0` (Monday) occurred **only** in the test window — 31.2 percent of test rows carried a value the model had never seen, silently inheriting an adjacent day's correction through bin placement. The gain was an artifact, not learned structure. The reported 166.6 s is the honest figure.

Fallback, enforced in code rather than documented: the agency's unmodified prediction is served when no artifact exists, when the model failed to beat its best baseline at training time, or when input falls outside the trained range. The baseline is the production default; the model is an override that must earn its place on every retrain.

8. Evaluation and validation evidence

8.1 Acceptance checks

`evaluation/run_acceptance_checks.py` verifies properties against the **actual data the pipeline produced**, distinct from unit tests which verify functions on imagined inputs.

Check	Result
`null_zero_discrimination`	408,149 rows compared against raw protobuf, **0 violations**
`contract_validation`	245,701 valid, 0 invalid
`grain_uniqueness`	6,046 arrivals, 6,046 distinct keys
`idempotent_resolution`	two independent runs, bit-for-bit identical
`provenance_complete`	0 missing fields
`event_time_not_ingest_time`	0 collisions with processing time

The first is the strongest artifact: it re-parses the raw protobuf and compares field *presence* directly against decoded output, rather than trusting the decoder's account of itself. Across 408,149 rows — 199,675 genuinely absent, 10,693 explicit zeros, 197,781 real values — zero violations.

Every check carries a row-count floor and reports `UNMEASURED` rather than `PASS` when handed too little data. A check that passes on an empty table is worse than no check: it reports green while measuring nothing, and continues reporting green after an upstream break empties its input.

A seventh check reports `INFO` rather than `PASS`. It found no loop-route revisits in the sample window, therefore asserted nothing; counting it as green would inflate the tally with a check that measured nothing. The behaviour it would have tested is asserted directly in the unit tests instead.

8.2 Unit tests

83 tests, covering the absent-versus-zero invariant, contract validation, resolver semantics, aggregation statistics, connection recovery and secret redaction, and 13 tests dedicated to ML leakage defences.

8.3 Pipeline throughput

Stage	Volume	Time
Replay producer	1,668,118 rows, 0 dead-lettered	77 s
Arrival resolver	38,701 positions to 6,046 arrivals	16 s
Full `make demo`	cold start to passing checks	2 min 15 s

9. Review path

Non-cloud, locally runnable, no API key required.

```
make venv && source .venv/bin/activate
make demo
```

This starts Kafka in Docker, creates topics, replays 39 minutes of archived feed data, derives arrivals, produces metrics, and runs the acceptance checks. It completes in approximately 2 minutes 15 seconds from cold.

Expected output ends with `passed=6 failed=0 unmeasured=0 info=1`, with results in `outputs/` and evidence in `evaluation/`.

Included sample data: `data/replay_sample/` — 39 polls spanning a contiguous 39-minute window (2026-08-06, 14:52–15:32), 13 MB of raw protobuf with a manifest.

Three properties are deliberate. The sample is **raw, not decoded**, so the reviewer exercises the decoder — the most correctness-critical component — rather than bypassing it. It is **contiguous, not randomly sampled**, because arrival detection requires consecutive observations. It is **gap-checked**, because a hole produces missed arrivals indistinguishable from an operator that does not report them.

Cleanup: `make kafka-down` stops and removes the containers. No cloud resources are provisioned.

10. Limitations and failures

10.1 Limitations

Coverage. Approximately 21 percent of stop events are captured, limited by the 60 requests/hour quota. Derived arrivals are biased late and over-represent long-dwell stops (Section 6.2), so reported bias is an upper bound.

Scope. One operator (SF Muni); six days of collection; no seasonal, holiday, or weather variation.

Model coverage. 95.8 percent of training data falls in hours 14–19, because the machine hosting the archiver sleeps overnight. The model is effectively an afternoon and evening model.

No confidence intervals. The model emits point estimates.

In-memory join. The aggregator joins across the full replay window in memory, which is honest for a bounded 39-minute replay but would require a windowed join with watermarks at production scale.

No Schema Registry. Compatibility is enforced by Pydantic contracts and a version stamp rather than centrally, so a producer could still publish a breaking change.

10.2 Failures encountered, and what they taught

A mislabelled partition column. A directory named `service_dt=` was populated with the UTC poll date, filing every poll between 17:00 and midnight under the following day — seven hours daily, including all of PM peak. Caught by comparing directory names against the dates actually inside the files. The correct fix was not "use the right date" but the realisation that a payload spans multiple service dates and therefore cannot be partitioned by service date at all.

A statistical claim that was wrong. The poll interval was initially described as "plus or minus 60 seconds of noise". Measuring dwell times disproved this: the real effects are a coverage limit, a one-sided late bias, and a selection bias — three different things, only one of which that phrasing described, and the correction materially weakens the headline claim.

A silent duplicate-counting bug. Replaying data twice doubled every sample count while leaving every mean, median, and percentile identical. The output looked entirely correct while `n` misreported. Caught by tearing down Kafka and re-running from a clean state.

An API key leaked into a log. The HTTP library embeds query parameters in exception messages, so failed fetches wrote the full URL — including the key — to disk. Caught by a pre-push secret scan. Now redacted at source and covered by tests.

A misleading green check. An acceptance check reported `PASS` while asserting nothing, because the sample contained no instances of the case it tested.

The common property is instructive: **every one of these produced output that looked correct**, and several produced *better*-looking numbers than the correct version. None crashed. Reviewing the code by reading it would have caught almost none of them. What caught them was running against real data and checking output against independent measurement.

10.3 Deviations from the proposal

Both disclosed deliberately.

A plain-Python consumer replaces Spark Structured Streaming. The proposal listed this as a sanctioned fallback; it was taken up front rather than discovered late. Delta Lake, the SCD2 schedule dimension, dbt, Dagster, and Kubernetes consequently fall outside the delivered scope and remain on the project roadmap.

Prediction accuracy replaces schedule-based on-time performance. Computing true OTP requires GTFS-Static schedule data and an as-of join against a slowly-changing dimension. The delivered metric exercises the identical streaming path and requires no additional data source.

11. Next steps

1. **Request a rate-limit increase.** Faster polling attacks coverage, one-sided offset, and selection bias simultaneously — the single highest-value improvement available.
2. **GTFS-Static integration** for true on-time performance against published schedules, via an SCD2 schedule dimension and as-of join.
3. **Additional operators**, beginning with VTA, which showed strong resolver A availability.
4. **Geofence resolver (method B)** for the 13 operators where method A cannot fire, calibrated against method A where both are available.
5. **Confidence intervals** on model output.
6. **Windowed join with watermarks**, replacing the in-memory aggregation.

12. Contributions

[To be completed. Aatish Lobo and Borna [last name] — split to be documented prior to submission.]

13. References and artifacts

Artifact	Location
Source data documentation	`DATA_SOURCE.md`
AI usage disclosure	`AI_USAGE.md`
Onboarding guide	`docs/ONBOARDING.md`
Pitfall register (60 items)	`docs/PITFALLS.md`
Daily build reports	`docs/reports/`
Model training report	`ml/artifacts/training_report.json`
Acceptance check results	`evaluation/acceptance_report.json`
Produced metrics	`outputs/`

Data source: 511 SF Bay Open Data, Metropolitan Transportation Commission. Data provided by 511.org — http://www.511.org